```
            return store[Priority.HIGH.ordinal()].removeFirst();
        }
        if (!store[Priority.NORMAL.ordinal()].isEmpty()) {
            return store[Priority.NORMAL.ordinal()].removeFirst();
        }
        if (!store[Priority.LOW.ordinal()].isEmpty()) {
            return store[Priority.LOW.ordinal()].removeFirst();
        }
        return null;
    }

    public static void main(String[] args) {
        Person p1 = new Person("A1", "Jim", "Black", LocalDate.of(1961,
Month.MARCH, 1));
        Person p2 = new Person("A2", "Jack", "Blue", LocalDate.of(1967,
Month.MARCH, 1));
        Person p3 = new Person("A3", "Jim", "Green", LocalDate.of(1971,
Month.MARCH, 1));
        Person p4 = new Person("A4", "Mary", "White", LocalDate.of(1961,
Month.MARCH, 1));
        Person p5 = new Person("A5", "Ann", "Red", LocalDate.of(1967,
Month.MARCH, 1));
        Person p6 = new Person("A6", "Kelly", "Pink", LocalDate.of(1971,
Month.MARCH, 1));
        MyPriorityQueue pQueue = new MyPriorityQueue();
        pQueue.add(Priority.LOW, p1);
        pQueue.add(Priority.LOW, p2);
        pQueue.add(Priority.NORMAL, p3);
        pQueue.add(Priority.NORMAL, p4);
        pQueue.add(Priority.HIGH, p5);
        pQueue.add(Priority.HIGH, p6);
        Person p;
        while ((p = pQueue.get()) != null) {
            System.out.println(p);
        }
    }

}
```

Κώδικας 12.21 *Η κλάση MyPriorityQueue*

Όπως φαίνεται στον κώδικα 12.21, η κύρια δομή της κλάσης MyPriorityQueue είναι ένας πίνακας από LinkedList. Η κάθε λίστα του πίνακα φιλοξενεί αντικείμενα τύπου Person. Η Priority είναι απαριθμήσιμος τύπος με τρείς τιμές, LOW, NORMAL και HIGH.

Στον δημιουργό της MyPriorityQueue, δημιουργείται καταρχάς ο πίνακας store με σταθερό μέγεθος 3, δηλαδή ένα στοιχείο για κάθε δυνατή προτεραιότητα. Για κάθε στοιχείο του πίνακα δημιουργείται μια LinkedList την οποία χειριζόμαστε σαν δομή FIFO.

Η add λαμβάνει δύο παραμέτρους, την προτεραιότητα και τον ασθενή. Από την προτεραιότητα εντοπίζει την επιμέρους ουρά προτεραιότητας στο τέλος της οποίας προσθέτει τον ασθενή εφόσον δεν υπάρχει ήδη σε αυτήν την ουρά.

Η get ελέγχει τις επιμέρους ουρές προτεραιότητας. Αν βρει ουρά υψηλής προτεραιότητας, επιστρέφει τον ασθενή στην αρχή της ουράς και ταυτοχρόνως τον διαγράφει από την ουρά. Αν η ουρά υψηλής προτεραιότητας είναι άδεια, προχωράει και ελέγχει κατά σειρά την ουρά μεσαίας προτεραιότητας και την ουρά χαμηλής προτεραιότητας. Αν όλες οι ουρές είναι άδειες, επιστρέφει null. Επομένως, ένας τρόπος για να ελέγξουμε αν η MyPriorityQueue είναι άδεια είναι αν η get μας επιστρέψει null. Αυτόν ακριβως τον τρόπο αξιοποιούμε στην main καθώς αδειάζουμε το αντικείμενο pQueue.

12.8 Ασκήσεις προς Λύση

12.8.1 Διαχείριση χωρητικότητας στην κλάση MyHashMap

Στην κλάση MyHashMap έχουμε ορίσει τον πίνακα store να έχει σταθερό μέγεθος. Η επιλογή αυτή δεν είναι ιδανική. Ενδεχομένως ένα μέγεθος της τάξης των δεκάδων θέσεων να είναι ικανοποιητικό όσο οι εγγραφές που καταχωρούνται είναι σχετικά λίγες. Καθώς όμως αυξάνεται ο αριθμός των εγγραφών, όλο και περισσότερες εγγραφές αντιστοιχίζονται στην ίδια θέση του πίνακα, δηλαδή όλο και περισσότερα αντικείμενα Person καταχωρούνται στην ίδια λίστα. Προφανές αποτέλεσμα είναι η καθυστέρηση των λειτουργιών της MyHashMap. Η HashMap του πακέτου java.util λύνει αυτό το πρόβλημα με τη βοήθεια της εσωτερικής χωρητικότητας (capacity) και του συντελεστή φόρτωσης (load factor). Και τα δύο μεγέθη μπορεί να καθοριστούν κατά τη δημιουργία των αντικειμένων HashMap και ισχύει ο εξής κανόνας: Όταν ο αριθμός των εγγραφών στη δομή hash είναι μεγαλύτερος από το γινόμενο του συντελεστή φόρτωσης επί την εσωτερική χωρητικότητα, τότε η εσωτερική χωρητικότητα αυξάνεται αυτόματα έτσι ώστε να είναι περίπου διπλάσια από τον αριθμό των εγγραφών.

Προσθέστε ανάλογη συμπεριφορά στην κλάση MyHashMap.

12.8.2 Τραπεζική εφαρμογή

Η τράπεζα World Bank διατηρεί λογαριασμούς πελατών δύο τύπων, τους προθεσμιακούς και του ταμιευτηρίου. Κάθε λογαριασμός, ανεξαρτήτως τύπου, διαθέτει έναν αριθμό λογαριασμού και μπορεί να έχει έναν ή περισσότερους δικαιούχους. Επιπλέον, ένας πελάτης της τράπεζας μπορεί να είναι δικαιούχος σε έναν ή περισσότερους λογαριασμούς. Κάθε φορά που ένας πελάτης επιθυμεί να κάνει κατάθεση ή ανάληψη χρημάτων θα πρέπει να προσδιορίζει τον αριθμό λογαριασμού του που θα κινηθεί. Η τράπεζα ελέγχει και αν βρεθεί λογαριασμός με αυτόν τον αριθμό που έχει ως δικαιούχο τον συγκεκριμένο πελάτη, προχωράει στην κίνηση του λογαριασμού. Αν όμως δεν βρεθεί τέτοιος λογαριασμός, ρωτάει τον πελάτη αν επιθυμεί να του ανοίξει λογαριασμό. Σε περίπτωση αρνητικής απάντησης, το αίτημα του πελάτη ακυρώνεται. Αν όμως ο πελάτης συναινέσει στη δημιουργία λογαριασμού, τότε η τράπεζα ρωτά αν ο λογαριασμός θα είναι προθεσμιακός ή ταμιευτηρίου και προχωρά στη δημιουργία και κίνηση του λογαριασμού. Ο αριθμός λογαριασμού δημιουργείται από την τράπεζα και είναι μοναδικός για κάθε λογαριασμό της ίδιας τράπεζας.

Οι λογαριασμοί κινούνται χρεούμενοι ή πιστούμενοι, δηλαδή η κατάθεση χρημάτων προκαλεί πίστωση του λογαριασμού και η ανάληψη χρέωση. Όλες οι κινήσεις ενός λογαριασμού διατηρούνται αναλυτικά από την τράπεζα. Κάθε κίνηση πέραν του τύπου της, δηλαδή αν είναι χρεωστική ή πιστωτική, διατηρεί και την ημερομηνία κατά την οποία πραγματοποιήθηκε καθώς και μία περιγραφή. Το υπόλοιπο ενός λογαριασμού είναι το σύνολο των πιστώσεών του μείον το σύνολο των χρεώσεών του. Οι δε δικαιούχοι δεν δύνανται να αποσύρουν από λογαριασμό ποσό μεγαλύτερο από το υπόλοιπό του.

Η τράπεζα έχει τη δυνατότητα να παράγει αναφορά για όλους τους λογαριασμούς ή αν ζητηθεί από πελάτη να παράγει αναφορά ειδικά για τους λογαριασμούς του συγκεκριμένου πελάτη. Για κάθε λογαριασμό που περιλαμβάνεται στην αναφορά της τράπεζας εμφανίζονται ο αριθμός λογαριασμού, ο τύπος του, οι δικαιούχοι του, οι κινήσεις του και το υπόλοιπό του σύμφωνα με το υπόδειγμα που ακολουθεί.

```
2 Προθεσμιακός, Δικαιούχοι:  John Black Mary Smith
2019-10-20, Πίστωση, 1000.0, Καταθεση
2020-08-17, Πίστωση, 1000.0, Καταθεση
2020-02-24, Χρέωση, 500.0, Ανάληψη
2019-11-29, Χρέωση, 300.0, Ανάληψη
Υπόλοιπο = 1200.0
```

Να δημιουργηθεί εφαρμογή που

1. Δημιουργεί μία τράπεζα
2. Δημιουργεί μια σειρά από πελάτες
3. Δημιουργεί μια σειρά από λογαριασμούς. Κάποιοι θα πρέπει να είναι προθεσμιακοί και κάποιοι ταμιευτηρίου. Κάποιοι θα πρέπει να έχουν έναν δικαιούχο και κάποιοι περισσότερους από έναν.
4. Οι πελάτες να προβούν σε καταθέσεις και αναλήψεις. Οι κινήσεις των λογαριασμών να γίνουν με τυχαία ημερομηνία που να περιλαμβάνεται στις τελευταίες 1000 ημέρες πριν την ημερομηνία του συστήματος.

5. Η τράπεζα να εκδώσει αναφορά για το σύνολο των λογαριασμών της.
6. Η τράπεζα να εκδώσει αναφορά κατόπιν αιτήματος συγκεκριμένου πελάτη.

Για την αναπαράσταση των ποσών να χρησιμοποιηθεί η κλάση BigDecimal.

Υποδείξεις

- Καταρχάς προσπαθήστε να εντοπίσετε τις κλάσεις αυτής της εφαρμογής. Συνήθως οι κλάσεις αντιστοιχούν σε ουσιαστικά μέσα στο κείμενο των απαιτήσεων, στην περίπτωσή μας στην εκφώνηση της άσκησης.
- Προσπαθήστε να βρείτε τις ενέργειες στις οποίες προβαίνει η κάθε κλάση και κωδικοποιήστε τες ως συναρτήσεις. Συνήθως, οι ενέργειες αντιστοιχούν σε ρήματα στο κείμενο των απαιτήσεων.
- Φροντίστε ώστε η διεπαφή χρήστη να υποστηρίζεται αποκλειστικά από ειδική κλάση εφοδιασμένη με τις κατάλληλες στατικές μεθόδους.

12.8.3 Δημιουργία μοναδικών στιγμιότυπων

Η κλάση Alpha διαθέτει δύο μέλη-δεδομένα τύπου String, το id και το description. Κάθε αντικείμενο Alpha προσδιορίζεται μονοσήμαντα από το id του. Η εφαρμογή στην οποία αξιοποιείται η Alpha απαιτεί να μην είναι δυνατή η δημιουργία δύο στιγμιότυπων με το ίδιο id ακόμη και αν αναφέρονται στο ίδιο αντικείμενο. Να υλοποιηθεί η κλάση Alpha.

Υπόδειξη

Ορίστε ένα στατικό HashMap<String, Alpha> στην κλάση Alpha. Κάντε τον δημιουργό της ιδιωτικό. Προσθέστε μία συνάρτηση public Alpha create(String id, String description) που ελέγχει αν αντικείμενο Alpha με αυτό το id υπάρχει ήδη στο HashMap. Αν το βρει το επιστρέφει, αν όχι το δημιουργεί, το προσθέτει στο HashMap και το επιστρέφει.

Βιβλιογραφία

[1] D. Liang, Εισαγωγή στον Προγραμματισμό Java, 10η. Θεσσαλονίκη: Τζιόλας.

[2] E. Freeman, E. Robson, B. Bates, and K. Sierra, Head First Design Patterns, 1st edition. Sebastopol, CA: O'Reilly Media, 2004.

Κεφάλαιο 13

Σύνοψη
Στην ενότητα αυτήν παρουσιάζεται αναλυτικά η κληρονομικότητα, δηλαδή το δεύτερο από τα θεμελιώδη χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου ενώ γίνεται εισαγωγή και στο τρίτο θεμελιώδες χαρακτηριστικό, τον πολυμορφισμό. Εξηγείται η δυνατότητα επανορισμού των κληρονομούμενων μεθόδων, ο ρόλος των λέξεων κλειδιών final και protected στο πλαίσιο της κληρονομικότητας, παρουσιάζεται η γονική κλάση Object, ο τελεστής instanceof και οι αφηρημένες κλάσεις. Συζητούνται κριτήρια επιλογής μεταξύ κληρονομικότητας και σύνθεσης.

Προαπαιτούμενη γνώση
Η ενσωμάτωση στο αντικειμενοστρεφές μοντέλο με Java και οι βασικές προκαθορισμένες κλάσεις, δηλαδή οι String, StringBuilder, ArrayList, LinkedList, Stack, PriorityQueue, HashSet, HashMap, HashTree, LocalDate, Localtime, Period και Duration.

Λέξεις κλειδιά
Κληρονομικότητα (Inheritance), Πολυμορφισμός (Polymorphism), Γονική κλάση (parent class), Παράγωγη κλάση (derived class), Αφηρημένη κλάση (abstract class).

13 Κληρονομικότητα και Πολυμορφισμός

Όπως έχουμε αναφέρει στην ενότητα 10, η κληρονομικότητα μας παρέχει τη δυνατότητα να συσχετίζουμε τις κλάσεις με σχέσεις ιεραρχίας και πιο συγκεκριμένα με σχέσεις προγόνου-απογόνου. Επάνω στην κληρονομικότητα βασίζεται το άλλο πανίσχυρο χαρακτηριστικό του αντικειμενοστρεφούς μοντέλου, ο πολυμορφισμός. Με τον πολυμορφισμό μπορούμε να διαχειριζόμαστε αντικείμενα μιας κλάσης απογόνου με αναφορές τύπου κάποιας προγονικής κλάσης ενώ ταυτόχρονα απολαμβάνουμε της προστασίας του συστήματος ελέγχου των τύπων. Στη συνέχεια παρουσιάζονται τα δύο αυτά χαρακτηριστικά και αναλύονται οι ιδιότητές τους και τα πλεονεκτήματά τους.

13.1 Η Κληρονομικότητα

Έστω η κλάση Root του κώδικα 13.1.

```
public class Root {
    private int id;
    private String Description;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getDescription() {
        return Description;
    }

    public void setDescription(String Description) {
        this.Description = Description;
    }

    @Override
    public int hashCode() {
        int hash = 7;
```

```
        hash = 89 * hash + this.id;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Root other = (Root) obj;
        return this.id==other.id;
    }

    @Override
    public String toString() {
        return id + ", " + Description;
    }
}
```

Κώδικας 13.1 *Η κλάση Root*

```
public class LeafA extends Root {
    public static void main(String[] args) {
        LeafA cA=new LeafA();
        System.out.println("getters: "+cA.getId()+" "+cA.getDescription());
        cA.setDescription("LeafA Object");
        System.out.println("toString: "+cA);
    }
}
```

Κώδικας 13.2 *Η κλάση LeafA κληρονομεί όλα τα χαρακτηριστικά της Root*

Στον κώδικα 13.2 καταρχάς δηλώνουμε ένα αντικείμενο της LeafA. Στη συνέχεια καλούμε την getId και την getDescription. Ωστόσο τέτοιες συναρτήσεις δεν έχουμε ορίσει στην LeafA. Παρόλα αυτά, ο κώδικας τρέχει καθώς η LeafA έχει στη διάθεσή της τις getId και getDescription που έχει κληρονομήσει από την Root. Το ίδιο ισχύει και για την setDescription αλλά και για την toString που καλείται αυτομάτως στην τελευταία γραμμή. Η έξοδος του κώδικα 13.2 έχει ως εξής:

```
getters: 0 null
toString: 0, LeafA Object
```

Αρχικά, η getId επιστρέφει 0 και η getDescription επιστρέφει null. Η LeafA εκτός από τις συναρτήσεις κληρονομεί και τις μη στατικές μεταβλητές της Root, το id και το description. Οι μεταβλητές αυτές δεν έχουν αρχικοποιηθεί στην Root και έτσι έχουν τις εξ' ορισμού τιμές ως μεταβλητές στιγμιότυπου. Στη συνέχεια, με την κλήση της setDescription δίνουμε την τιμή LeafA Object στην description που παρουσιάζει η toString της τελευταίας γραμμής.

Η κλάση Root ονομάζεται γονική κλάση (parent class) ή βασική κλάση (Base class) γιατί από αυτήν κληρονομεί τα χαρακτηριστικά της η LeafA. Η δε LeafA ονομάζεται και κλάση-παιδί (child class) ή παράγωγη κλάση (derived class).

Αν προσθέσουμε τον ακόλουθο δημιουργό στην Root

```
public Root() {
     System.out.println("Root constructor");
```

```
        this.id = 1;
        this.Description = "Parent";
    }
```

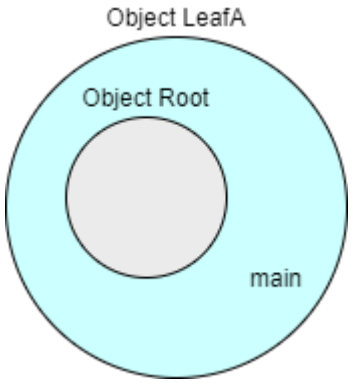
τότε η έξοδος του κώδικα 13.2 θα μεταβληθεί ως εξής:

```
Root constructor
getters: 1 Parent
toString: 1, ChildA Object
```

Βλέπουμε δηλαδή πώς κλήθηκε ο δημιουργός της κλάσης Root, παρότι δεν δημιουργήσαμε σαφώς ένα αντικείμενο Root, και έδωσε τις τιμές 1 και Parent στις μεταβλητές id και description, αντίστοιχα.
Αν προσθέσουμε και έναν δημιουργό με παραμέτρους στην Root, π.χ.

```
public Root(int id, String Description) {
    this.id = id;
    this.Description = Description;
}
```

μπορούμε εύκολα να διαπιστώσουμε ότι η έξοδος του κώδικα 13.2 δεν θα μεταβληθεί. Αν όμως διαγράψουμε τον δημιουργό χωρίς παραμέτρους της Root και αφήσουμε μόνο τον δημιουργό με παραμέτρους θα διαπιστώσουμε πως η LeafA δεν μεταγλωττίζεται.
Πώς εξηγείται αυτή η συμπεριφορά; Κάθε αντικείμενο της παράγωγης κλάσης LeafA περικλείει ένα αντικείμενο την γονικής κλάσης Root όπως δείχνει το σχήμα 13.1.



Σχήμα 13.1 *Η παράγωγη κλάση περικλείει ένα αντικείμενο της γονικής.*

Πέραν των συναρτήσεων και μεταβλητών που κληρονομεί η παράγωγος κλάση από τη γονική, μπορεί να διαθέτει επιπλέον συναρτήσεις ή και μεταβλητές. Συγκεκριμένα, η κλάση LeafA διαθέτει μια μέθοδο main. Όπως φαίνεται στο σχήμα 13.1, τα αντικείμενα της παράγωγης περιέχουν ένα αντικείμενο της γονικής και τα μέλη, δηλαδή την main στο παράδειγμά μας, που ανήκουν αποκλειστικά στην παράγωγη. Κατά την κατασκευή των αντικειμένων LeafA, ο δημιουργός της κλάσης καλεί τον δημιουργό της γονικής προκειμένου να κατασκευάσει το εσωτερικό αντικείμενο Root. Όταν η γονική διαθέτει δημιουργό χωρίς παραμέτρους, τότε η κλήση του μπορεί να αυτοματοποιηθεί. Όταν όμως η γονική κλάση δεν διαθέτει δημιουργό χωρίς παραμέτρους, τότε η κλήση του δημιουργού θα πρέπει να πραγματοποιηθεί σαφώς, καθώς απαιτείται να δοθούν τιμές στις παραμέτρους του. Η κλήση αυτή γίνεται στον δημιουργό της παράγωγης με τη βοήθεια της λέξης-κλειδί super, ως εξής:

```
public LeafA(int id, String Description) {
    super(id, Description);
}
```

Επομένως σε αυτήν την περίπτωση, η παράγωγη κλάση υποχρεωτικά πρέπει να διαθέτει έναν δημιουργό ο οποίος να είναι σε θέση να καλέσει τον δημιουργό της γονικής και να δώσει τιμές στις παραμέτρους του. Στην

τυπική περίπτωση, οι παράμετροι του δημιουργού της γονικής είναι και παράμετροι του δημιουργού της παράγωγης. Αν ωστόσο η παράγωγη έχει και άλλα μέλη που πρέπει να αρχικοποιηθούν στον δημιουργό, τότε αυτό θα γίνει μετά την κλήση του δημιουργού της γονικής. Με άλλα λόγια, η κλήση του δημιουργού της γονικής απαιτείται να είναι η πρώτη εντολή στον δημιουργό της παράγωγης. Εκείνο που πρέπει να τονιστεί είναι πως οι δημιουργοί δεν κληρονομούνται, δηλαδή δεν μπορούμε να καλέσουμε τον δημιουργό γονικής για να κατασκευάσουμε αντικείμενο παράγωγης. Αυτό είναι αναγκαία συνέπεια ώστε η κλήση ενός δημιουργού να κατασκευάζει με σαφήνεια αντικείμενο συγκεκριμένης κλάσης.
Με τη βοήθεια του νέου δημιουργού της, η παράγωγη μπορεί να δημιουργήσει αντικείμενα όπως επιδεικνύει ο κώδικας 13.3:

```
    public static void main(String[] args) {
        LeafA cA=new LeafA(4,"LeafA");
        System.out.println("getters: "+cA.getId()+" "+cA.getDescription());
        cA.setDescription("ChildA Object");
        System.out.println("toString: "+cA);
    }
```

Κώδικας 13.3 *Δημιουργία παράγωγων αντικειμένων όταν η γονική διαθέτει μόνο δημιουργούς με παραμέτρους*

Η έξοδος του κώδικα 13.3 έχει ως εξής:

```
getters: 4 LeafA
toString: 4, ChildA Object
```

Παρότι η LeafA κληρονομεί όλες τις μεταβλητές και συναρτήσεις της Root, η πρόσβαση στα ιδιωτικά μέλη της Root δεν είναι δυνατή από την LeafA, δηλαδή ο κώδικας

```
System.out.println(cA.id);
```

προκαλεί λάθος μεταγλώττισης.
Ο κώδικας 13.4 επιδεικνύει μια άλλη κλάση, επίσης παράγωγη της Root.

```
public class LeafB extends Root {
    private double value;

    public LeafB(double value, int id, String Description) {
        super(id, Description);
        this.value = value;
    }

    public double getValue() {
        return value;
    }

    public void setValue(double value) {
        this.value = value;
    }

    public static void main(String[] args) {
        LeafB leafB=new LeafB(120,5,"LeafB");
        System.out.println(leafB.getId()+" "+leafB.getDescription()+"
"+leafB.getValue());
    }
}
```

Κώδικας 13.4 *Παράγωγη κλάση με επιπλέον χαρακτηριστικά*

Η κλάση LeafB κληρονομεί από την κλάση Root, διαθέτει όμως μια επιπλέον μεταβλητή στιγμιότυπου, την value. Για τη διαχείριση της μεταβλητής, η κλάση ορίζει τους κατάλληλους ρυθμιστές και αναγνώστες. Ο

δημιουργός της κλάσης καλεί τον δημιουργό της γονικής για να κατασκευάσει το εσωτερικό αντικείμενο και στη συνέχεια δίνει τιμή στην επιπλέον μεταβλητή value.
Η έξοδος της main του κώδικα 13.4 έχει ως εξής:

```
5 LeafB 120.0
```

Η κληρονομικότητα δεν περιορίζεται σε ένα επίπεδο. Αντίθετα, δεν υπάρχει κανένας περιορισμός ούτε κατά βάθος ούτε κατά πλάτος. Για παράδειγμα, η κλάση των ηλεκτρικών συσκευών, μπορεί να έχει πολλούς απογόνους στο ίδιο επίπεδο, π.χ. υπολογιστές, πλυντήρια, τηλεοράσεις, ραδιόφωνα, αλλά και πολλούς απογόνους κάθετα, π.χ. ο υπολογιστής μπορεί να έχει ως απογόνους τον επιτραπέζιο υπολογιστή και τον φορητό υπολογιστή, ενώ ο φορητός υπολογιστής μπορεί να έχει απογόνους τον υπολογιστή χειρός (calculator) και την ταμπλέτα (tablet). Στην πράξη, συνήθως διαμορφώνονται ιεραρχίες κλάσεων που συνδέονται μεταξύ τους με κληρονομικότητα. Οι ιεραρχίες κληρονομικότητας συναντώνται εκτεταμένα και στις βιβλιοθήκες της Java.

13.1.1 Επανορισμός κληρονομούμενων μεθόδων

Ποια θα είναι όμως η έξοδος αν μετατρέψουμε την main της LeafB όπως δείχνει ο κώδικας 13.5;

```
public static void main(String[] args) {
    LeafB leafB=new LeafB(120,5,"LeafB");
    System.out.println(leafB);
}
```

Κώδικας 13.5 *Η main καλεί την toString της γονικής*

Εδώ εμφανίζουμε κατευθείαν ένα αντικείμενο LeafB. Όπως γνωρίζουμε, σε αυτές τις περιπτώσεις καλείται η μέθοδος toString για να επιστρέψει την αναπαράσταση του αντικειμένου με τη μορφή συμβολοσειράς. Ωστόσο, στην LeafB δεν έχουμε ορίσει μέθοδο toString. Επομένως αυτή κληρονομείται από την Root. Η toString της Root βεβαίως δεν συμπεριλαμβάνει τη μεταβλητή value. Συνεπώς, η έξοδος της main είναι:

```
5, LeafB
```

Αν μια μέθοδος της γονικής δεν καλύπτει τις ανάγκες της παράγωγης, τότε η μέθοδος αυτή μπορεί να επανοριστεί (override) στο επίπεδο της παράγωγης. Αν, επομένως, η toString της Root δεν μας καλύπτει για την LeafB, τότε θα την επανορίσουμε στην LeafB. Κατά τον επανορισμό της μεθόδου, μπορούμε εφόσον μας διευκολύνει να χρησιμοποιήσουμε τη μέθοδο της γονικής και να προσθέσουμε την επιπλέον απαιτούμενη λειτουργικότητα, όπως δείχνει ο κώδικας 13.6.

```
@Override
public String toString() { //Κώδικας 13.6
    return super.toString() + ", " + value;
}
```

Κώδικας 13.6 *Η toString επανοριζόμενη στην παράγωγη κλάση LeafB*

Προκειμένου να καλέσουμε τη μέθοδο της γονικής από την παράγωγη, χρησιμοποιούμε και πάλι τη λέξη κλειδί super αλλά με διαφορετικό συντακτικό από ότι στον δημιουργό. Ο κώδικας 13.5 τώρα έχει την ακόλουθη έξοδο:

```
5, LeafB, 120.0
```

Η σειρά @Override είναι ένα σχόλιο (annotation) προς τον μεταγλωττιστή με το οποίο πληροφορείται ότι η μέθοδος που ακολουθεί είναι επανορισμός κληρονομούμενης μεθόδου. Το σχόλιο αυτό δεν είναι υποχρεωτικό, είναι όμως χρήσιμο. Για παράδειγμα, αν επιχειρώντας να επανορίσουμε τη μέθοδο toString, δηλώσουμε κατά λάθος μέθοδο asString(), τότε προσθέτουμε μια νέα μέθοδο και δεν επανορίζουμε την toString. Ως αποτέλεσμα, τα αντικείμενα LeafB θα συνεχίσουν να εμφανίζονται χωρίς την τιμή της value.

Ένα άλλο παράδειγμα. Αν ορίσουμε μια μέθοδο String toString(double value), τότε έχουμε υπερφορτώσει τη μέθοδο toString, δεν την έχουμε όμως επανορίσει. Με την @Override, ο μεταγλωττιστής μπορεί να ανιχνεύσει τέτοιου είδους σφάλματα και να μας προστατεύσει από αυτά.

Προσέξτε πως δεν τίθεται θέμα επανορισμού στην παράγωγη των ιδιωτικών μεθόδων της γονικής, καθώς για αυτές δεν επιτρέπεται η πρόσβαση στην παράγωγη κλάση. Επομένως, αν ορίσουμε μια μέθοδο στην παράγωγη με ακριβώς την ίδια διεπαφή με ιδιωτική μέθοδο της γονικής, έχουμε ορίσει απλώς μια νέα μέθοδο. Επιπλέον, κατά τον επανορισμό μιας μεθόδου, έχουμε τη δυνατότητα να ορίσουμε ευρύτερο προσδιοριστή προσπέλασης. Για παράδειγμα, μια μέθοδος που στη γονική έχει πρόσβαση πακέτου, μπορεί στην παράγωγη να έχει δημόσια πρόσβαση.

Οι στατικές μέθοδοι και μεταβλητές δεν συνδέονται με τα αντικείμενα αλλά με την ίδια την κλάση. Έτσι, τα στατικά μέλη δεν κληρονομούνται ούτε και επανορίζονται. Αν οριστεί σε παράγωγη κλάση στατική μέθοδος με ίδια ταυτότητα με στατική μέθοδο της γονικής, τότε πρόκειται για δύο διαφορετικές μεθόδους. Η διάκρισή τους από τον μεταγλωττιστή γίνεται με βάση το όνομα της κλάσης.

13.1.2 Final και protected

Αν επιθυμούμε να μην επιτρέπουμε στις παράγωγες κλάσεις να έχουν τη δυνατότητα να επανορίσουν μια μέθοδο της γονικής, τότε μπορούμε να χαρακτηρίσουμε αυτήν τη μέθοδο ως final, όπως δείχνει το παρακάτω απόσπασμα κώδικα:

```
public final int getId() {
     return id;
}
```

Αν πάλι επιθυμούμε να αποτρέψουμε τη δημιουργία παράγωγων κλάσεων, τότε μπορούμε να χαρακτηρίσουμε ως final ολόκληρη την κλάση, όπως δείχνει το απόσπασμα κώδικα που ακολουθεί:

```
public final class LeafA extends Root {}
```

Η κλάση LeafA είναι παράγωγη της κλάσης Root, αλλά δεν μπορεί να δημιουργηθεί κλάση παράγωγη της LeafA.
Είπαμε στην ενότητα 13.1 πως τα μέλη id και description είναι ιδιωτικά στην κλάση Root και επομένως δεν επιτρέπεται η απευθείας πρόσβαση σε αυτά από την παράγωγη LeafA. Αν επιθυμούμε τα μέλη αυτά να προστατεύονται από τρίτους, ωστόσο, οι παράγωγες κλάσεις να έχουν πρόσβαση και σε αυτά, τότε μπορούμε να τα χαρακτηρίσουμε ως protected, όπως δείχνει το ακόλουθο απόσπασμα κώδικα:

```
protected int id;
protected String Description;
```

Χρησιμοποιούμε δηλαδή τον προσδιοριστή προσπέλασης protected στη θέση του private. Ας σημειωθεί πως protected μπορεί να χαρακτηρισθεί και μια κλάση, εμφωλευμένη ή μη.

13.1.3 Κληρονομικότητα ή σύνθεση;

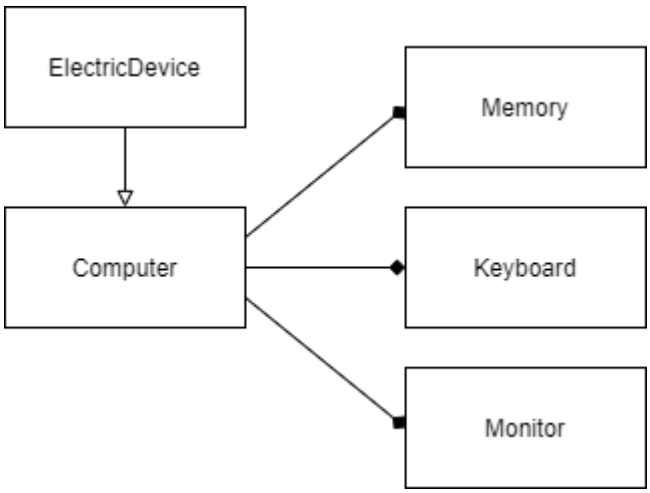
Δύο είναι οι βασικές σχέσεις μεταξύ των κλάσεων που θα μας απασχολήσουν στην ενότητα αυτήν, η σχέση “είναι ένα” (is a) και η σχέση “έχει ένα” (has a). Ένα ραδιόφωνο είναι μία ηλεκτρική συσκευή. Επομένως, ένα ραδιόφωνο διαθέτει όλα τα χαρακτηριστικά και λειτουργίες μιας ηλεκτρικής συσκευής, π.χ. μια ηλεκτρική συσκευή μπορεί να είναι συνδεδεμένη στο ρεύμα ή όχι, το ίδιο ισχύει και για ένα ραδιόφωνο. Ένα αυτοκίνητο είναι ένα όχημα. Ένα όχημα μπορεί να κινηθεί, το ίδιο και ένα αυτοκίνητο. Είναι λοιπόν σαφές πως η ηλεκτρική συσκευή και το ραδιόφωνο συνδέονται με σχέση κληρονομικότητας, δηλαδή το ραδιόφωνο ως κλάση μπορεί να κληρονομεί χαρακτηριστικά και λειτουργίες της ηλεκτρικής συσκευής. Το ίδιο ισχύει και μεταξύ των κλάσεων που αναπαριστούν το όχημα και το αυτοκίνητο.

Από την άλλη πλευρά, ένα αυτοκίνητο έχει έναν κινητήρα. Επίσης, έχει μηχανισμό ταχυτήτων. Η σχέση “έχει ένα” υποδηλώνει σύνθεση (composition) και όχι κληρονομικότητα, δηλαδή η κλάση αυτοκίνητο μπορεί να περιλαμβάνει ένα μέλος της κλάσης που αναπαριστά τους κινητήρες και ένα μέλος της κλάσης που αναπαριστά τους μηχανισμούς ταχυτήτων.

Ας το δούμε με ένα παράδειγμα. Έστω η κλάση Computer αποτελεί μια απλή προσομοίωση ενός υπολογιστή. Αυτού του τύπου οι υπολογιστές υποστηρίζουν μόνο τρεις εντολές, την keyboardToMem που

διαβάζει από το πληκτρολόγιο ένα String και το μεταφέρει στη μνήμη, την MemToMonitor, που εμφανίζει στην οθόνη τα περιεχόμενα της μνήμης και την clearMem που αρχικοποιεί τη μνήμη. Επομένως, ο υπολογιστής πρέπει να έχει μνήμη, πληκτρολόγιο και οθόνη. Ταυτόχρονα όμως ο υπολογιστής μας είναι και μια ηλεκτρική συσκευή και όπως όλες οι ηλεκτρικές συσκευές μπορεί να είναι συνδεδεμένος στο ρεύμα ή όχι.

Από την περιγραφή του παραδείγματός μας είναι φανερό πως ο υπολογιστής συνδέεται με τις κλάσεις που αναπαριστούν τη μνήμη, την οθόνη και το πληκτρολόγιο με τη σχέση "έχει ένα", ενώ με τις ηλεκτρικές συσκευές με τη σχέση "είναι ένα". Το παρακάτω διάγραμμα σε UML (Unified Modelling Language) [1] δείχνει τις σχέσεις μεταξύ των κλάσεων.



Σχήμα 13.2 *Σχέσεις σύνθεσης και κληρονομικότητας*

Στο σχήμα 13.2 το βελάκι με τριγωνική απόληξη χωρίς γέμισμα συμβολίζει κληρονομικότητα, δηλαδή ο Computer κληρονομεί από την ElectricDevice. Αντίθετα, το βελάκι με συμπαγή απόληξη σε σχήμα ρόμβου συμβολίζει σύνθεση, δηλαδή ο Computer έχει Memory, Keyboard και Monitor.

Ας αναπτύξουμε τώρα τις κλάσεις. Καταρχάς, χρειαζόμαστε την κλάση ElectricDevice. Η κλάση αυτή διαθέτει μία κύρια λειτουργία η οποία τη συνδέει ή την αποσυνδέει από το ρεύμα και μια λειτουργία που ελέγχει αν είναι ή όχι συνδεδεμένη στο ρεύμα, όπως δείχνει ο κώδικας 13.7.

```
public class ElectricDevice  {

    private boolean powerOn;

    public boolean isPowerOn() {
        return powerOn;
    }

    public void setPowerOn(boolean powerOn) {
        this.powerOn = powerOn;
    }
}
```

Κώδικας 13.7 *Η κλάση ElectricDevice*

Ας δούμε τώρα τις κλάσεις Keyboard, Monitor και Memory.

```
public class Keyboard {

     public String read() {
          return "ABCD";
     }
}
```

Η κλάση Keyboard διαθέτει μια απλή μέθοδο read που για απλότητα επιστρέφει μια συμβολοσειρά. Μπορείτε να αναπτύξετε την read ώστε πράγματι να λαμβάνει μια είσοδο από το πληκτρολόγιο και να την επιστρέφει. Όπως και να έχει, δεν επηρεάζονται οι έννοιες που παρουσιάζουμε εδώ.

```
public class Memory {

     private String content=null;

     public String getContent() {
          return content;
     }

     public void setContent(String content) {
          this.content = content;
     }

     public boolean isEmpty() {
          return content==null;
     }
}
```

Η κλάση Memory έχει τρεις λειτουργίες. Μπορεί να αποθηκεύσει μια τιμή τύπου String, να επιστρέψει την αποθηκευμένη τιμή και να ελέγξει αν υπάρχει ή όχι αποθηκευμένη τιμή.
Η κλάση Monitor μπορεί απλώς να εμφανίσει μια τιμή String.

```
public class Monitor {

     public void display(String data) {
          System.out.println(data);
     }
}
```

Τέλος, στον κώδικα 13.8 παρουσιάζεται η κλάση Computer:

```
public class Computer extends ElectricDevice {

    private final Keyboard keyboard = new Keyboard();
    private final Memory memory = new Memory();
    private final Monitor monitor = new Monitor();

    void keyboardToMem() {
        if (!isPowerOn()) {
            return;
        }
        String data = keyboard.read();
        memory.setContent(data);
    }

    void memToMonitor() {
        if (!isPowerOn() || memory.isEmpty()) {
            return;
        }
        monitor.display(memory.getContent());
    }

    void clearMem() {
```